

جامعة أم القرى
UMM AL-QURA UNIVERSITY

Database Project Report

Cinema Management Database System

Maareb Felemban 44611014
Gheem ghazawi 44611429
Dr.Najwa Samargandi

11 May 2026

Table of Contents

- Team Responsibilities ……..... Page 4

Part 1: Conceptual and Logical Design

- Identify Entities, Attributes, and Relationships ..... Page 4
- ER Diagram……..... Page 5
- Relational Schema……..... Page 5

Part 2: Database Implementation

- Database Creation ..... Page 6
- Tables Creation & Data Types……Page6
- Constraints (PK, FK, UNIQUE, CHECK) .....…….Page 6
- Sample Data Insertion ..... Page 6

Part 3: SQL Queries and Report

- Basic SELECT Queries ..... Page 7
- Aggregate Function Queries ..... Page 7
- Join Query ..... Page 7
- Subquery Analysis ..... Page 7
- View for Reporting..... Page 7

Appendices & Closing

- **Screenshots Appendix ..... Page 8**
  - ***ER Diagram & Schema Results ..... Page 8***
  - ***Query Execution Results ..... Page 9***
- **Conclusion ..... Page 13**

Team Responsibilities

- **Maareb Ehab Felemban:** Database schema design, SQL implementation (Part 3), and project coordination.
- **Gheem Ghazawi:** Documentation, ER diagram design, and sample data collection

Part 1: Conceptual and Logical Design

Identify Entities, Attributes, and Relationships:

- **Cinema:** (cinema_id, name, location)
- **Screen:** (screen_id, screen_number, capacity, cinema_id)
- **Movie:** (movie_id, title, duration, rating, release_date)
- **Actor:** (actor_id, name)
- **Customer:** (customer_id, name, email, phone)
- **Ticket:** (ticket_id, seat_number, price, customer_id, show_id)
- **Show:** (show_id, show time, show date, movie_id,screen_id)
- **Relationships:** Cinema has Screens (1:M), Movie has Actors (M:N), Screens have Shows (1:M), and Customers buy Tickets (1:M).

ER Diagram:

1. **Overview:** The Entity-Relationship (ER) Diagram represents the conceptual design of the **Cinema Management System**. It illustrates how data is structured and how different entities interact with each other.
2. **Entities and Attributes:**

- **Cinema:** Represents the physical theater locations, storing names and addresses.
- **Movie:** Contains film details such as titles, duration, and age ratings.
- **Customer:** Stores information about users who interact with the system.
- **Ticket:** Acts as a link between customers, shows, and pricing.

3. **Relationships:**

- **One-to-Many (1:M):** One Cinema can have multiple Screens, and one Screen can host many Shows.
- **Many-to-Many (M:N):** A Movie can have multiple Actors, and an Actor can star in many Movies.
- **One-to-Many (1:M):** A Customer can purchase multiple Tickets, but each Ticket is linked to only one Customer.

Relational Schema:

- Cinema (**cinema_id**, name, location)
- Screen (**screen_id**, screen_number, capacity, *cinema_id*)
- Movie (**movie_id**, title, duration, rating, release_date)
- Movie_Actor (**movie_id**, **actor_id**)
- Show_Table (**show_id**, show_time, show_date, *movie_id*, *screen_id*)
- Customer (**customer_id**, name, email, phone)
- Ticket (**ticket_id**, seat_number, price, *customer_id*, *show_id*)

Part 2: Database Implementation

Database Creation:

```
1    -- 1. Create a new database
2    CREATE DATABASE CinemaDB;
3    USE CinemaDB;
```

Tables Creation & Data Types:

- All tables were defined with specific data types to ensure storage efficiency and data accuracy.
- **INT:** Used for all ID columns (e.g., cinema_id, movie_id) to allow for fast indexing and numerical operations.
- **VARCHAR:** Used for names, titles, and locations (e.g., name in Cinema is VARCHAR(100)).
- **DECIMAL(10, 2):** Specifically used for the price in the Ticket table to handle currency values precisely.
- **DATE & TIME:** Used in Movie and Show_Table to manage release dates and screening schedules accurately.

Constraints:

- **Primary Keys (PK):** Defined for every table to uniquely identify records.
- **Foreign Keys (FK):** Used to link tables together (e.g., linking Screen to Cinema).
- **NOT NULL:** Applied to essential fields like names and dates to ensure data integrity.
- **UNIQUE:** Applied to the customer's email to prevent duplicate accounts.
- **CHECK:** Used in the Screen table to ensure capacity > 0.

Sample Data:

- Five sample records were inserted into each table to test the system functionality.
- Data includes real-world examples such as 'Vox Cinema', 'Inception', and customer details like 'Ahmed Ali'.

Part 3 SQL Queries and Report

Basic SELECT Query 1: This query is designed to retrieve the contact information (email) for a specific customer named 'Ahmed Ali'.

Basic SELECT Query 2: This query filters the movie library to list titles and durations for films that run longer than 150 minutes, which is useful for scheduling long screenings.

Aggregate Function Query 1: By using the COUNT function and GROUP BY clause, this query calculates the total number of screens assigned to each individual cinema.

Aggregate Function Query 2: The AVG function is utilized here to determine the average price of all tickets issued in the database.

Join Query: **Purpose:** This query performs an inner join between the Ticket and Customer tables to provide a report that links seat numbers and ticket IDs to the specific customers who bought them.

Subquery: **Purpose:** A subquery is implemented to answer a specific operational question: "Which movie titles are scheduled for shows on the date 2026-05-10?".

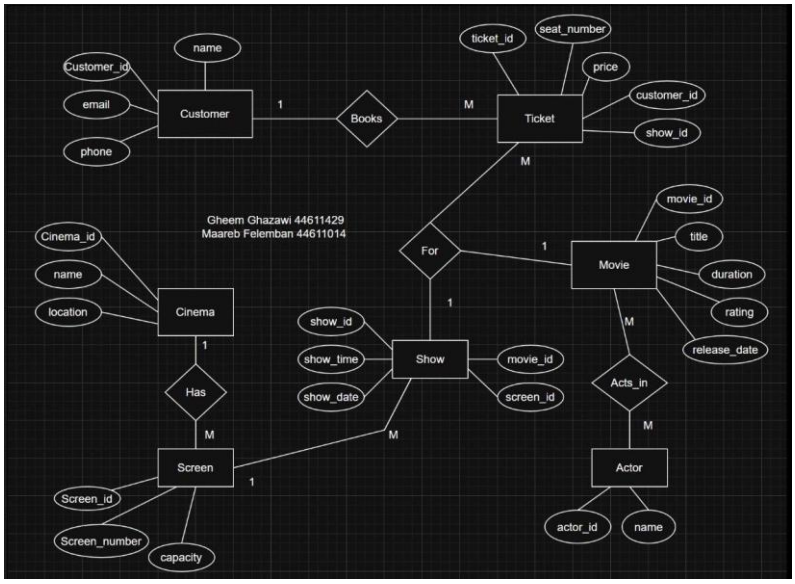
Logic: The inner query identifies the movie_id from the Show_Table, and the outer query uses these IDs to fetch the corresponding movie titles.

View:**Purpose:** A database view called Daily_Show_Report was created to simplify reporting by merging data from the Movie and Show_Table.

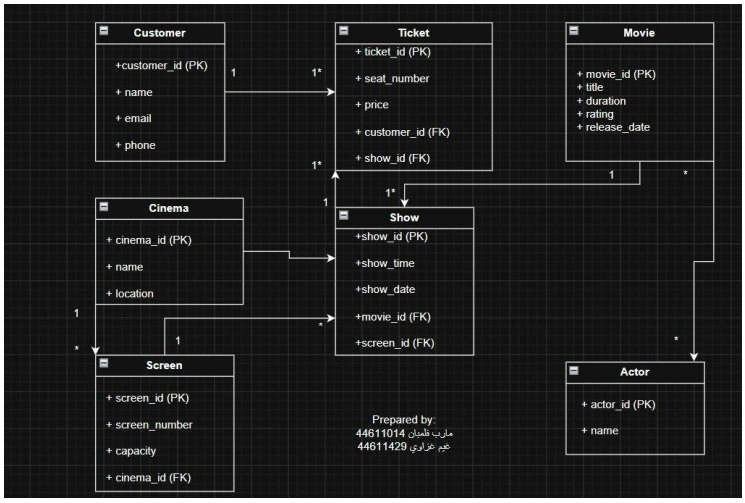
Benefits: It allows staff to quickly view movie titles alongside their dates and times without writing complex joins repeatedly.

Screenshots (Appendix)

Screenshot 1: ER Diagram



Screenshot 2: Relational Schema



Screenshot 3: Database Creation

```sql
-- 1. Create a new database
CREATE DATABASE CinemaDB;
USE CinemaDB;
```

Screenshot 4: Tables Structure

```sql
-- 1. Create a new database
CREATE DATABASE CinemaDB;
USE CinemaDB;

-- 2. Create tables with appropriate data types and constraints (PK, FK, UNIQUE, NOT NULL, CHECK)
CREATE TABLE Cinema (
    cinema_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    location VARCHAR(255) NOT NULL
);

CREATE TABLE Screen (
    screen_id INT PRIMARY KEY,
    screen_number INT NOT NULL,
    capacity INT CHECK (capacity > 0),
    cinema_id INT,
    FOREIGN KEY (cinema_id) REFERENCES Cinema(cinema_id)
);

CREATE TABLE Movie (
    movie_id INT PRIMARY KEY,
    title VARCHAR(150) NOT NULL,
    duration INT NOT NULL,
    rating VARCHAR(10),
    release_date DATE
);

CREATE TABLE Actor (
    actor_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE Movie_Actor (
    movie_id INT,
    actor_id INT,
    PRIMARY KEY (movie_id, actor_id),
    FOREIGN KEY (movie_id) REFERENCES Movie(movie_id),
    FOREIGN KEY (actor_id) REFERENCES Actor(actor_id)
);

CREATE TABLE Show_Table (
    show_id INT PRIMARY KEY,
    show_time TIME NOT NULL,
    show_date DATE NOT NULL,
    movie_id INT,
    screen_id INT,
    FOREIGN KEY (movie_id) REFERENCES Movie(movie_id),
    FOREIGN KEY (screen_id) REFERENCES Screen(screen_id)
);
```

```sql
CREATE TABLE Customer (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(20)
);

CREATE TABLE Ticket (
    ticket_id INT PRIMARY KEY,
    seat_number VARCHAR(10) NOT NULL,
    price DECIMAL(10, 2) NOT NULL,
    customer_id INT,
    show_id INT,
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
    FOREIGN KEY (show_id) REFERENCES Show_Table(show_id)
);
```

Screenshot 5&6: Sample Data (Table 1) with Basic SELECT Query 1 Result

```sql
-- 1. TWO Basic SELECT (columns + WHERE conditions)
-- Query 1: Get names and emails of customers named 'Ahmed Ali'
SELECT name, email
FROM Customer
WHERE name = 'Ahmed Ali';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	email
Ahmed Ali	ahmed@email.com

Customer 1

Output

Action Output

#	Time	Action	Message
18	01:25:30	INSERT INTO Ticket VALUES (601, 'A1', 50.00, 501, 401), (602, 'B5', 65.00, 502, 4...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
19	01:25:43	SELECT name, email FROM Customer WHERE name = 'Ahmed Ali' LIMIT 0, 1000	1 row(s) returned

Screenshot 7&8: Sample Data (Table 2) with Basic SELECT Query 2 Result

```sql
WHERE name = 'Ahmed Ali';

-- Query 2: Get movies with a duration longer than 150 minutes
SELECT title, duration
FROM Movie
WHERE duration > 150;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

title	duration
The Dark Knight	152
Interstellar	169
Avatar	162

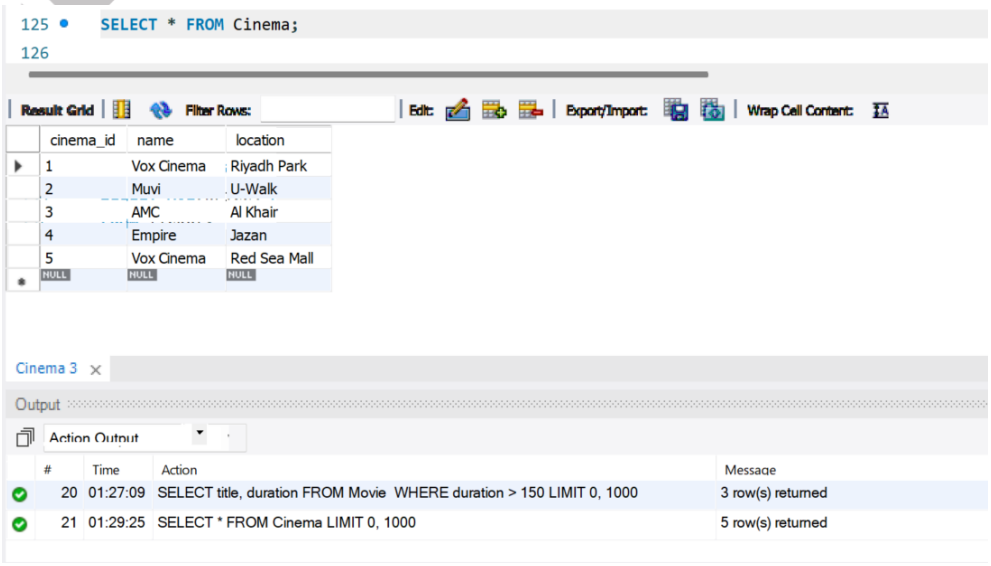
Movie 2

Output

Action Output

#	Time	Action	Message
19	01:25:43	SELECT name, email FROM Customer WHERE name = 'Ahmed Ali' LIMIT 0, 1000	1 row(s) returned
20	01:27:09	SELECT title, duration FROM Movie WHERE duration > 150 LIMIT 0, 1000	3 row(s) returned

Screenshot 9: Sample Data (Table 3)



```
125 •  SELECT * FROM Cinema;
126
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

cinema_id	name	location
1	Vox Cinema	Riyadh Park
2	Muvi	U-Walk
3	AMC	Al Khair
4	Empire	Jazan
5	Vox Cinema	Red Sea Mall
NULL	NULL	NULL

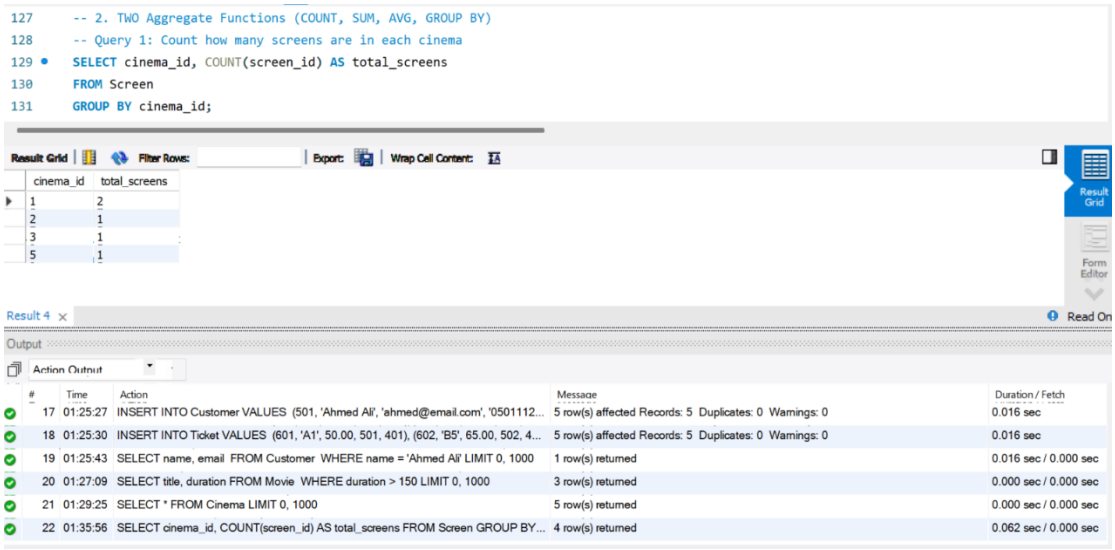
Cinema 3

Output

Action Output

#	Time	Action	Message
20	01:27:09	SELECT title, duration FROM Movie WHERE duration > 150 LIMIT 0, 1000	3 row(s) returned
21	01:29:25	SELECT * FROM Cinema LIMIT 0, 1000	5 row(s) returned

Screenshot 10: Aggregate Function Result



```
127     -- 2. TWO Aggregate Functions (COUNT, SUM, AVG, GROUP BY)
128     -- Query 1: Count how many screens are in each cinema
129 •   SELECT cinema_id, COUNT(screen_id) AS total_screens
130     FROM Screen
131     GROUP BY cinema_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cinema_id	total_screens
1	2
2	1
3	1
5	1

Result Grid | Form Editor

Result 4

Output

Action Output

#	Time	Action	Message	Duration / Fetch
17	01:25:27	INSERT INTO Customer VALUES (501, 'Ahmed Ali', 'ahmed@email.com', '0501112...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.016 sec
18	01:25:30	INSERT INTO Ticket VALUES (601, 'A1', 50.00, 501, 401), (602, 'B5', 65.00, 502, 4...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.016 sec
19	01:25:43	SELECT name, email FROM Customer WHERE name = 'Ahmed Ali' LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
20	01:27:09	SELECT title, duration FROM Movie WHERE duration > 150 LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
21	01:29:25	SELECT * FROM Cinema LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
22	01:35:56	SELECT cinema_id, COUNT(screen_id) AS total_screens FROM Screen GROUP BY...	4 row(s) returned	0.062 sec / 0.000 sec

Screenshot 11: JOIN Query Result

```
137     -- 3. ONE Join between tables
138     -- Get ticket details along with the customer names who bought them
139 •   SELECT Ticket.ticket_id, Ticket.seat_number, Customer.name
140     FROM Ticket
141     JOIN Customer ON Ticket.customer_id = Customer.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ticket_id	seat_number	name
601	A1	Ahmed Ali
602	B5	Sara Khalid
603	C3	Omar Hassan
604	D10	Laila Fahad
605	A2	Noura Saad

Result 5

Output

Action Output

#	Time	Action	Message	Duration / Fetch
20	01:27:09	SELECT title, duration FROM Movie WHERE duration > 150 LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
21	01:29:25	SELECT * FROM Cinema LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
22	01:35:56	SELECT cinema_id, COUNT(screen_id) AS total_screens FROM Screen GROUP BY...	4 row(s) returned	0.062 sec / 0.000 sec
23	01:39:01	SELECT Ticket.ticket_id, Ticket.seat_number, Customer.name FROM Ticket JOIN C...	5 row(s) returned	0.016 sec / 0.000 sec

Screenshot 12: Subquery Result

```
143     -- 4. ONE Subquery to answer a specific question
144     -- Find movies that have shows scheduled on '2026-05-10'
145 •   SELECT title
146     FROM Movie
147     WHERE movie_id IN (SELECT movie_id FROM Show_Table WHERE show_date = '2026-05-10');
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ticket_id	seat_number	name
601	A1	Ahmed Ali
602	B5	Sara Khalid
603	C3	Omar Hassan
604	D10	Laila Fahad
605	A2	Noura Saad

Result 5

Output

Action Output

#	Time	Action	Message	Duration / Fetch
20	01:27:09	SELECT title, duration FROM Movie WHERE duration > 150 LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
21	01:29:25	SELECT * FROM Cinema LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
22	01:35:56	SELECT cinema_id, COUNT(screen_id) AS total_screens FROM Screen GROUP BY...	4 row(s) returned	0.062 sec / 0.000 sec
23	01:39:01	SELECT Ticket.ticket_id, Ticket.seat_number, Customer.name FROM Ticket JOIN C...	5 row(s) returned	0.016 sec / 0.000 sec

Screenshot 13: View Result

```sql
-- 5. View to generate a simple report
-- Create a view that shows daily show details (Movie Title, Date, and Time)
CREATE VIEW Daily_Show_Report AS
SELECT Movie.title, Show_Table.show_date, Show_Table.show_time
FROM Show_Table
JOIN Movie ON Show_Table.movie_id = Movie.movie_id;

-- To display the report from the view:
SELECT * FROM Daily_Show_Report;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

title	show_date	show_time
Inception	2026-05-10	18:00:00
The Dark Knight	2026-05-10	21:00:00
Interstellar	2026-05-11	15:00:00
Avatar	2026-05-11	19:30:00
Toy Story	2026-05-12	10:00:00

Daily_Show_Report 6 ×

Output

Action Output

#	Time	Action	Message	Duration / Fetch
23	01:39:01	SELECT Ticket.ticket_id, Ticket.seat_number, Customer.name FROM Ticket JOIN C...	5 row(s) returned	0.016 sec / 0.000 sec

Conclusion

In conclusion, this project successfully demonstrated the design and implementation of a comprehensive Cinema Management System using MySQL. The process began with building a solid conceptual foundation through the ER Diagram and Relational Schema, ensuring that all entities—such as Cinemas, Movies, and Customers—were logically connected.

By implementing the database, we effectively managed data through various SQL techniques, including:

- Data Integrity: Enforcing primary and foreign keys to maintain accurate relationships.
- Advanced Querying: Using Joins, Subqueries, and Aggregate functions to extract meaningful business insights, such as daily show schedules and sales summaries.
- Optimization: Creating a Database View to simplify complex reporting tasks.

Overall, this project provided practical experience in bridging the gap between theoretical database design and real-world application, resulting in a functional system capable of managing a modern cinema environment efficiently.







جامعة أم القرى

UMM AL-QURA UNIVERSITY